Sustainable Computing: Informatics and Systems 40 (2023) 100906



Contents lists available at ScienceDirect

Sustainable Computing: Informatics and Systems

journal homepage: www.elsevier.com/locate/suscom





Multi-exit DNN inference acceleration for intelligent terminal with heterogeneous processors

Jinghui Zhang [a], Weilong Xin [b], Dingyang Lv [a], Jiawei Wang [c], Guangxing Cai [a], Fang Dong [a,*]

[a] School of Computer Science and Engineering, Southeast University, Nanjing, 211189, China
[b] College of Software Engineering, Southeast University, Nanjing, 211189, China
[c] School of Cyber Science and Engineering, Southeast University, Nanjing, 211189, China

ARTICLE INFO

Keywords:
Heterogeneous processor
Multi-exit DNN
Inference acceleration
Intelligent terminal device

ABSTRACT

Recently, there has been a burgeoning popularity in the deployment of deep learning vision applications upon terminal devices. However, as the number of layers in deep neural networks (DNNs) and structural complexity increase, although the performance of DNN in handling computer vision tasks has become powerful, model inference tasks on computation resource constrained intelligent terminal devices are frequently incapable of meeting latency requirement. A commonly adopted solution to inference acceleration presents multi-exit DNNs to reduce latency via the provision of early exits. However, existing methods do not fully utilize the potential of heterogeneous processors (GPU/CPU) on intelligent terminal devices to cooperatively accelerate multi-exit DNN inference in parallel. Furthermore, the impact of complex image and video input on multi-exit DNNs, as well as the effects of different power consumption modes on processors within intelligent terminal devices, remain inadequately explored. To address these issues, we comprehensively considered the computing performance of heterogeneous processors in different power consumption modes, the structure and characteristic of multi-exit DNNs in inference acceleration, and proposed the **C**ollaborative **I**nference **A**cceleration mechanism for intelligent terminal with **H**eterogeneous **P**rocessors (CIAHP). CIAHP includes a deep neural network computation time prediction model and a multi-exit DNN task allocation algorithm with heterogeneous processors. Our experiments demonstrate that CIAHP performs multi-exit DNN inference 2.31× faster than CPU alone, and is 1.23× faster than GPU alone when processing complex image samples.

1. Introduction

In recent years, deep learning has achieved significant breakthroughs in a variety of fields. The widespread use of deep neural networks (DNNs) has facilitated great improvements in performance for computer vision (CV) tasks such as image classification [1], video analytics [2], object detection [3,4] and virtual/augmented reality [5]. As shown in Fig. 1, different from training task which update model parameters with train set and category labels, DNNs in inference tasks interpret input samples and generate results without updating their parameters. Even lightweight Convolutional Neural Networks like MobileNet [6], explicitly designed for mobile devices, still necessitate a substantial computational budget, requiring 569 million floating-point operations. Moreover, the challenge becomes more acute when considering real-time applications. Real-time performance is paramount, particularly in scenarios such as autonomous driving [7], where captured scenes or images are subject to rapid changes. As a result, low latency and high precision must be ensured when performing the inference task of DNN.

Several investigations have employed device-edge-cloud synergistic inference to meet real-time requirements of inference tasks in image classification and object detection [8–11]. However, the emergence of privacy policies, such as the General Data Protection Regulation (GDPR) [12], has restricted the upload of data to cloud servers, including images and videos, making the aforementioned approaches are no longer applicable under the premise of privacy preservation. Meanwhile, advancements in the Internet of Things (IoT) [7] and DNN have facilitated the development of intelligent terminal devices, including smartphones and autonomous cars [13], enabling local inference capabilities gradually emerging as a new trend. For example, driverless cars need to perform local inference in order to detect and avoid obstacle [14]. However, numerous resource-constrained intelligent terminal device, (e.g. Jetson Nano, Raspberry Pi) lack the computing capacity required to perform real-time inference for deep learning tasks [15]. To address this challenge, several methods have been developed to accelerate DNN inference on resource-constrained terminal devices. A

* Corresponding author.
E-mail address: fdong@seu.edu.cn (F. Dong).

https://doi.org/10.1016/j.suscom.2023.100906
Received 27 February 2023; Received in revised form 14 July 2023; Accepted 8 August 2023
Available online 22 August 2023
2210-5379/© 2023 Elsevier Inc. All rights reserved.

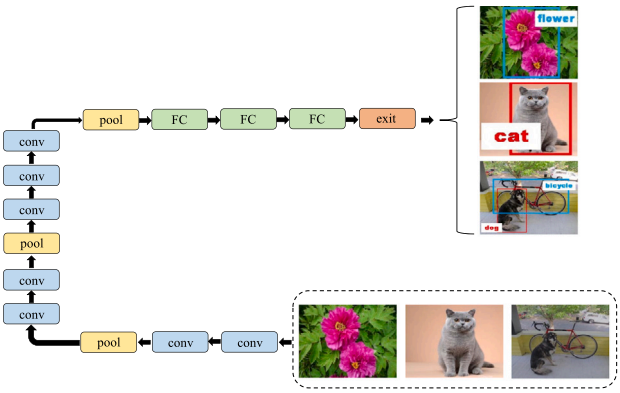


Fig. 1. The object detection inference task of DNN.

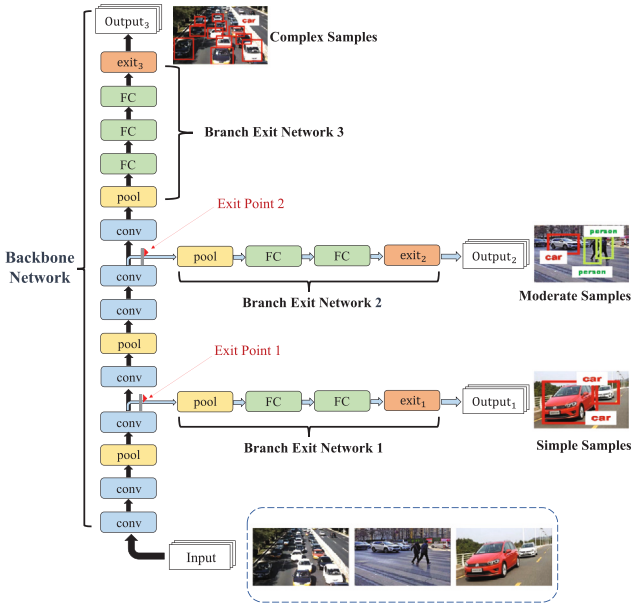


Fig. 2. The illustration of a multi-exit DNN.

prevalent approach involves the use of multi-exit DNNs on intelligent terminal devices, as proposed by Laskaridis et al. (2020) and Dong et al. (2022) [16,17]. As shown in Fig. 2, by integrating multiple classifiers consisting mainly of fully connected network layers as branch exit networks to the shallow neural network layers, a portion of the inference tasks can achieve high confidence above the threshold via shallow exits and complete early with an acceptable accuracy. Consequently, the DNN inference task is completed ahead of schedule, enabling lower latency for images classification tasks.

Currently, most intelligent terminal devices employ heterogeneous processors, including multiple CPUs and GPUs [18]. As depicted in Fig. 3, when the CPU and GPU of intelligent terminal devices process computational tasks for certain fully connected layers, there is negligible disparity in computing efficiency [19,20]. Both processors provide similar resource efficiencies when executing inference process of fully connected network layers. Accordingly, assigning the execution of inference of fully connected network layers to the CPU does not result in a significant increase in latency. Based on the aforementioned finding, inference process for branch exit networks and backbone networks can be performed simultaneously on the GPU and the CPU to improve overall computing efficiency on intelligent terminal devices. In the meantime, there are often multiple power consumption modes on intelligent terminal devices. Different power consumption modes result

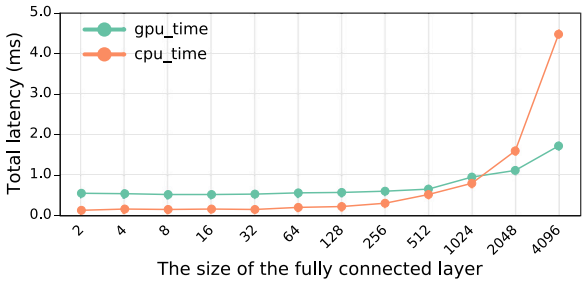


Fig. 3. The computational latency of the GPU/CPU on different size of fully connected layer. ($x \times 512 \times 10$).

in various clock rates and latency for intelligent terminal devices, it has also led to a change in processor performance. However, traditional multi-exit DNNs fail to leverage the potential benefits of heterogeneous processors on the device when solely relying on the GPU. This leads to higher inference latency, particularly for complex inputs, as the CPU remains idle during inference process for branch exit networks or backbone networks executed by the GPU. Furthermore, most of the current research fails to take into account the impact of changes in power consumption modes for terminal devices. In order to improve the inference efficiency of terminal devices, the inference execution needs to be sensibly allocated between the GPU and CPU for each network layer in different power consumption modes.

To achieve such objective, this paper intends to exploit heterogeneous processors (CPU/GPU) on the intelligent terminal device to jointly process the multi-exit DNN inference task. Specifically, we aim to design a multi-exit DNN inference acceleration mechanism utilizing heterogeneous processors of intelligent terminal devices to reduce the inference latency and alleviate the high latency problem caused by multi-exit DNNs when confronted with complex samples. In the meantime, the traditional approach to multi-exit DNN inference acceleration suffers from many limitations: (1) The existing research on multi-exit DNNs focuses primarily on the design of branch structures and confidence thresholds [21], but fails to address the issue of increased latency resulting from complex inputs. (2) Research on inference acceleration on intelligent terminal devices has overlooked the performance characteristics of multiple processors [8], resulting in suboptimal utilization of the computing performance of each processor. (3) Current efforts often ignore the real-time power consumption mode of intelligent terminal devices [22], even though the performance of processors varies depending on the power consumption mode. In order to build a multi-exit DNN inference acceleration mechanism for heterogeneous processors, the inference latency of each network layer on different processors needs to be considered, as well as the impact on performance due to the different power consumption modes of terminal devices. Furthermore, the cost of intermediate data transfers between the GPU and CPU should also be taken into consideration.

We propose the **C**ollaborative **I**nference **A**cceleration mechanism for intelligent terminal with **H**eterogeneous **P**rocessors (CIAHP) in this paper, aimed at improving the computing efficiency of intelligent terminal devices, effectively reducing the latency of multi-exit DNN inference tasks. The main contributions of this paper are as follows:

- We construct a deep neural network computation time prediction model based on heterogeneous processors, which takes into account the power consumption mode of the intelligent terminal device and the parameter information of each neural layer.
- To design a multi-exit DNN task allocation algorithm with heterogeneous processors, we consider comprehensively the computing time of DNNs, the structure of multi-exit DNNs, the classification ability of each branch exit, and the size of the intermediate data generated by each network layer.

- A series of experiments were conducted to evaluate the performance of CIAHP on heterogeneous processors. The results demonstrated that CIAHP outperformed existing methods by achieving noteworthy reductions in inference latency, while concurrently maintaining high accuracy levels.

2. Related work

2.1. Acceleration technology based on heterogeneous processors

The rapid progressions in deep learning technology, alongside the extensive proliferation of intelligent terminal devices, have made it necessary to deploy deep learning applications on these devices. Regarding the hardware configuration of terminal devices, the prevailing approach currently focuses on accelerating GPU execution, such as NVIDIA's cuDNN [23]. Additionally, some devices offer domain-specific computing processors, like Google's Tensor Processing Unit (TPU) [24], which have demonstrated significant computational power for specific tasks like image analysis. In the meantime, terminal devices come equipped with heterogeneous processors (GPU/CPU) [18]. To make full use of the computing efficiency of all processors and satisfy the performance requirements of deep learning applications on these devices, numerous methods have been proposed to exploit the potential of heterogeneous processors on intelligent terminal devices for such applications.

EunJin et al. [25] proposed a TensorRT-based framework that supports a variety of optimization parameters to enhance the performance of deep learning applications on NVIDIA Jetson embedded platforms with heterogeneous processors. Yecheng et al. proposed DART framework [26], which ensures timing constraints for real-time DNN inference tasks while minimizing response time. It aims to improve the utilization of heterogeneous CPU and GPU cores to maximize the throughput of best-effort DNN application. The majority of these approaches utilize pipeline to increase the throughput of DNN tasks on the intelligent terminal device based on heterogeneous processors. Any change in the performance of a processor in pipeline will result in sub-optimal overall efficiency, especially for devices where the power consumption mode can change frequently. Joo et al. [27] introduced BAND, a mobile inference system that coordinates multi-DNNs workloads on heterogeneous processors. BAND pre-divides a DNN into a set of sub-graphs while taking into account operator dependency. At runtime, BAND dynamically selects a schedule from multiple possible schedules according to the target of a pluggable scheduling policy. This flexibility enables lower latency for applications involving multiple DNNs. Nevertheless, the applicability of the BAND framework is limited to multi-DNNs workflow scenarios, and it is not suitable for accelerating the inference task of a single DNN. Lane et al. [28] proposed DeepX, which employs CPUs and GPUs to co-process a DNN inference task. DeepX divides the deep learning model into multiple modules that are executed on the GPU and CPU according to the characteristics of each module. However, despite DeepX's consideration of collaborative processing on heterogeneous processors, its synchronous collaboration between CPUs and GPUs only utilizes one processor for a task at a time, leaving the rest of the processors idle and not fully leveraging their performance. At the same time, most of these approaches do not take into account the impact of the device's power consumption mode on the performance of the processors. This makes the processor less efficient at handling DNN inference tasks.

2.2. Acceleration technology based on multi-exit DNN

As the number of layers in deep neural networks increases, there is a generally observed enhancement in their capacity to learn and express features. However, this also leads to higher model complexity and increased computation time and energy consumption during forward propagation. To address this issue, Teerapittayanon et al. [29] proposed a novel multi-exit DNN architecture. By integrating multiple classifiers as exits between the neural network layers, a proportion of the sample can be exited early through a shallow exit. The inference process of a multi-exit DNN basically contains two primary components: the backbone and the branch. For the input image data, the multi-exit DNN first performs the inference computation of the backbone network. When the image data reaches the entry point of the branch exit network, the inference computation of the backbone network is suspended and the branch exit network is executed. The branch exit network outputs the classification results of images and provides a confidence level. If the confidence level reaches a threshold, the image data can exit early, otherwise it returns to the backbone network. This approach allows for the completion of the inference task ahead of schedule, ensuring timely completion of the image classification task.

Researchers have proposed the deployment of multi-exit DNNs in different locations within the "device-edge-cloud" in a distributed manner, thereby enabling collaborative inferences by multiple devices to reduce the latency of the model and network delay [16,17,30–34]. However, there are still several shortcomings in existing work on multi-exit DNNs. For example, when it comes to a complex sample input, the sample cannot exit from the shallow exits early. As a result, the device perform inference computation for all the backbone networks and branch exit networks. This not only fails to achieve the desired effect but also leads to unpredictable inference latency [35]. Thorough analysis of this scenario and the optimization of multi-exit DNN execution mode are pivotal in enhancing the efficiency of inference tasks.

The above methods, however, fail to fully utilize processors on the intelligent terminal device and neglect the coordination of heterogeneous processors. Currently, when multi-exit DNNs are applied to complex samples, the inference path covers all backbone and branch networks, resulting in an increase in inference latency [35]. Moreover, none of the above methods take into account the impact of different power consumption modes on computing performance.

3. Preliminaries

Based on terminal device with different power consumption modes, we first discusses the factors influencing the execution time of DNN inference and develop a DNN computation time prediction model based on heterogeneous processors. In order to maximize the efficiency of the CPU and GPU, we then analyzed the execution characteristics of DNNs on heterogeneous processors. Based on the time prediction model, we propose a multi-exit DNN inference acceleration algorithm, the details of which are elaborated in Section 4.

3.1. Deep neural network computation time prediction model

Several factors affect the computation time of the neural network on the intelligent terminal device, including the computational complexity of network layers, the power consumption mode of intelligent terminal device, and the type of processor. To illustrate how to predict computation time, we use a convolutional neural network (CNN) as an example.

CNN includes convolutional network layers, pooling network layers, and fully connected network layers. Research indicates that these three types of network layers account for a significant portion of the inference time of AlexNet when computed solely on GPUs (96.4%) or CPUs (95.7%) [36]. Therefore, the main objective is to optimize these three network layers. The floating-point arithmetic (FLOPs) of each network layer are calculated as Eq. (1), Eq. (2) and (3), where $FLOPs_CONV$, $FLOPs_POOL$, $FLOPs_FC$ are the calculation amount of convolutional layer, pooling layer and fully connected layer respectively.

$$FLOPs_CONV = [(C_{in} \times k_w \times k_h) + (C_{in} \times k_w \times k_h - 1)] \times C_{out} \times W \times H \tag{1}$$

Table 1
Notations of computation time prediction model.

Symbol	Description
e	power consumption mode of the intelligent terminal device
W, H	width and height of the feature map
C_{in}, C_{out}	number of input and output channels for network layer
k_w, k_h	width and height of the convolution kernel
p_w, p_h	width and height of the pooling kernel
F_w, F_h	number of input and output neurons of the fully connected layer
θ	parameter quantity information
cpu_mode	binary variable, 1 for CPU and 0 for GPU

$$FLOPs_POOL = C_{in} \times p_w \times p_h \tag{2}$$

$$FLOPs_FC = 2F_{in} \times F_{out}, \tag{3}$$

where, C is the number of channels, K is the size of the convolution kernel, and F is the number of neurons in the fully connected layer. It is assumed that the height and width of each kernel are generally same.

In addition to the FLOPs, factors that affect the inference time include: the power consumption mode, the type of processor and the parameter information of model. Even if the FLOPs of models are the same, the execution time of different processor types varies due to different power consumption modes. There are often multiple power consumption modes and two processor types on intelligent terminal devices. Power consumption modes can be broadly divided into low-power mode and high-power mode. Different power consumption modes result in various clock rates and latency for intelligent terminal devices. Different types of processors have different resource efficiency. Therefore, power consumption mode and processor type need to be considered in order to build the computation time prediction model. Table 1 lists the key notations used in the computation time prediction model.

According to the above analysis, the definition of the deep neural network computation time prediction model on heterogeneous processors is as follows:

$$y_i(f(C_{in}, C_{out}, F_{in}, F_{out}, k_w, p_w), e, \theta, cpu_mode), \tag{4}$$

where, e is the power consumption mode of the device, θ is the parameter information of the model, cpu_mode is a binary variable, 1 for CPU and 0 for GPU, f represents the computational workload of the neural network, which is determined by the number of network layers and the quantity of FLPOs per layer.

With prediction model, the execution times of the different network layers can be retrieved. These execution times will be used to determine the optimal allocation strategy for the network layers.

3.2. Preliminary analysis on CPU/GPU performance comparison

Initially, we use NVIDIA Jetson Nano as experimental equipment, CIFAR-10 [37] dataset and VGG16 [38] model. The latency differences between different processors are shown in Fig. 4. The gc_ratio indicates the ratio of the time overhead of the GPU to the time overhead of the CPU. When $0 < gc_ratio < 1$, the computing performance of the GPU is better and $gc_ratio > 1$ means the CPU is better. The $cpu_load = 0\%$ indicates that no other load tasks are currently running on the device. It can be observed that due to the multi-core parallel architecture of GPUs, their computational efficiency in convolutional layers is significantly higher than CPUs. Conversely, the computational efficiency of GPUs and CPUs in fully connected layers is not significantly different. The computational efficiency of CPUs is even twice as high as that of GPUs in some fully connected layers.

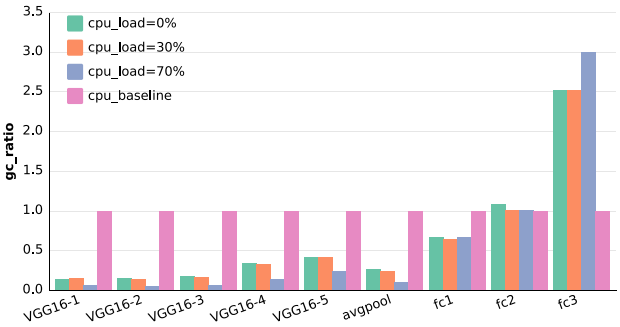


Fig. 4. The inference latency ratio of different processors at each network layer on Jetson Nano.

4. Methodology

Leveraging the computation time prediction model conceived in the preceding section, the objective of this section is to contrive an innovative methodology to efficiently distribute multi-exit DNN inference tasks amidst heterogeneous processors. This section is composed of three subsections, namely the multi-exit DNN model, problem formulation, and acceleration algorithm.

4.1. Multi-exit DNN model

Table 2 details the relevant parameters of the multi-exit DNN model. In the architectural design of the multi-exit DNN, auxiliary exit networks are integrated within the DNN, which is shown in Fig. 5. The architecture is divided into two distinctive divisions: the backbone and the ancillary branches. The backbone architecture is composed of n modules $L = l_1, l_2, l_3, \ldots, l_{n-1}, l_n$, with each module encompassing multiple network layers. With the exception of the l_n module, which predominantly comprises fully connected layers, the remaining modules primarily consist of convolutional and pooling layers. The ancillary branch exit network is interspersed amidst neighboring blocks, where each branch exit network is composed of a combination of pooling and fully connected layers. As represented in Fig. 5, the multi-exit DNN includes an aggregate of $n-1$ prospective exits, not accounting for the backbone exit. The model is capable of acquiring the confidence level of a sample at the branch exit network and providing the result of the sample with a predetermined probability in advance. The probability of a sample departing from the ith branch exit network is symbolized by p_i, and p_0 is an virtual exit set to zero. Eq. (5) designates μ_i as the probability that the input sample will abstain from exiting until the ith backbone network layer is executed:

$$\mu_i = \prod_{j=1}^{i} (1 - p_{j-1}). \tag{5}$$

The pertinent notations and their respective definitions are manifested in Table 3. Decision variables embody the task distribution across computational resources. Here, $X_{l_i} = [0, 1]$ and $X_{b_i} = [0, 1]$ signify that the ith backbone network layer and the ith branch exit network are allocated to the CPU, whereas $X_{l_i} = [1, 0]$ and $X_{b_i} = [1, 0]$ denote that these corresponding elements are designated to the GPU. It follows that $X_{l_i}[0] \oplus X_{l_i}[1] = 1$ and $X_{b_i}[0] \oplus X_{b_i}[1] = 1$. In light of the analysis derived from the previous section, it has been simplified thus: tasks associated with the convolutional layer are designated to the GPU, implying that $\forall i \in [1, n-1], X_{l_i} = [1, 0]$. Given the overlap of the nth branch exit network with the final backbone network layer, it holds that X_{l_n} is congruent to X_{b_n}.

The identification of the power consumption mode for intelligent terminal devices, coupled with the prediction of computation latency for heterogeneous processors, necessitate immediate consideration.

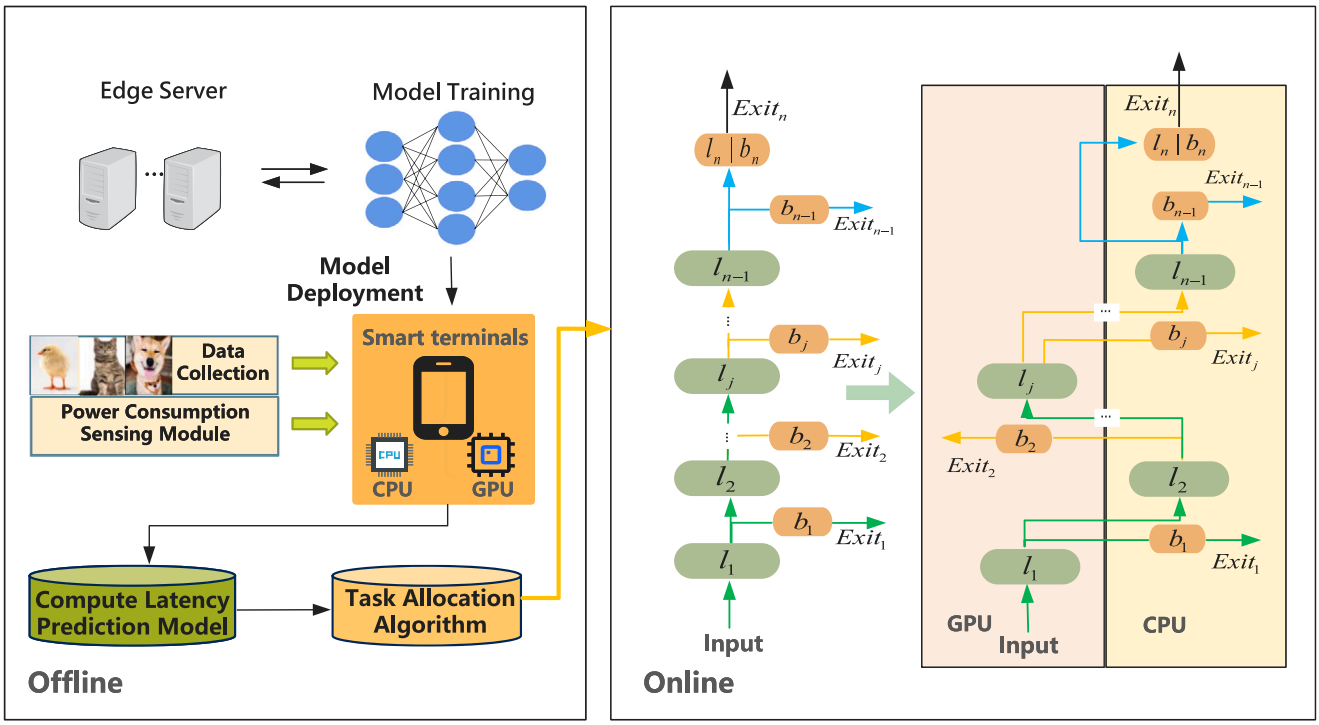


Fig. 5. The illustration of multi-exit DNNs inference acceleration mechanism based on heterogeneous processor.

Table 2
Notations of multi-exit DNN predicated on heterogeneous processing unit.

Symbol	Meaning
$l = l_1, l_2, \ldots, l_{n-1}, l_n$	backbone network layers
$B = b_1, b_2, \ldots, b_{n-1}, b_n$	branch exit networks
$P = p_0, p_1, p_2, \ldots, p_{n-1}, p_n$	exit probability of branch exit networks
$I_{b_i} = \{f(C_{in}^{b_i}, C_{out}^{b_i}, F_{in}^{b_i}, F_{out}^{b_i}, k_w^{b_i}, p_w^{b_i}), \theta^{b_i}\}$	information of the ith branch exit network layer
$I_{l_i} = \{f(C_{in}^{l_i}, C_{out}^{l_i}, F_{in}^{l_i}, F_{out}^{l_i}, k_w^{l_i}, p_w^{l_i}), \theta^{l_i}\}$	information of the ith backbone network layer
$t_{l_i}^{cpu}$	computation time of ith backbone network layer on CPU
$t_{l_i}^{gpu}$	computation time of ith backbone network layer on GPU
$t_{b_i}^{cpu}$	computation time of ith branch exit network on CPU
$t_{b_i}^{gpu}$	computation time of ith branch exit network on GPU
DS_i	size of intermediate data of the ith backbone network layer
BW_{pcie}	PCIe bandwidth between CPU and GPU
μ_i	execution probability of ith backbone network layer

Table 3
Notations of decision.

Symbol	Description
X_L	computing task allocation of all backbone network layers
X_{l_i}	computing task allocation of ith backbone network layer
X_B	computing task allocation of all branch exits
X_{b_i}	computing task allocation of ith branch exit

Consequently, we utilize the deep neural network computation time prediction model, conceived in Eq. (4), to ascertain computation latency of each network layer in different processors with different power consumption modes.

The distribution of computing tasks ought to consider not solely the computational latency but also the expense associated with transmitting intermediate data between heterogeneous processors. If DS_i denotes the size of the intermediate data engendered by the ith backbone network layer, the ensuing delay for DS_i transmission from GPU to CPU is characterized as: $t_{b_i}^{d2h} = \frac{DS_i}{BW_{pcie}}$, while the delay for the converse, from CPU to GPU, is: $t_{b_i}^{h2d} = \frac{DS_i}{BW_{pcie}}$. It is assumed by default that $t_{b_i}^{d2h} = t_{b_i}^{h2d} = t_{b_i}^{trans}$.

4.2. Problem formulation

Presuming the current inference task is undergoing execution at the ith backbone network layer and will conclude at the corresponding ith branch exit network, its optimal execution duration is depicted in Eq. (6). The function $g(L_i)$ embodies the cumulative execution duration under the optimal allocation strategy from the first network layer through to the $i-1$th network layer.

$$target_{b_i} = \min\{g(L_i) + t_{l_{i-1},b_i}^{trans} + t_{l_i} + t_{b_i}\} \tag{6}$$

$$t_{l_{i-1},b_i}^{trans} = t_{l_{i-1}}^{trans} + t_{b_i}^{trans} \tag{7}$$

$$t_{l_{i-1}}^{trans} = (X_{l_i}[0] \oplus X_{l_{i-1}}[0]) \times (t_{l_{i-1}}^{h2d} \times X_{l_{i-1}}[1] + t_{l_{i-1}}^{d2h} \times X_{l_{i-1}}[0]) \tag{8}$$

$$t_{b_i}^{trans} = (X_{l_i}[0] \oplus X_{b_i}[0]) \times (t_i^{h2d} \times X_{b_i}[1] + t_i^{d2h} \times X_{b_i}[0]), \tag{9}$$

herein, $t_{l_{i-1}}^{trans}$ denotes the intermediate data transfer overhead incurred between the $i-1$th backbone network layer and the ith backbone network layer. $t_{b_i}^{trans}$ signifies the intermediate data transfer overhead between the ith backbone exit point and the branch exit network. t_{l_i} represents the computation overhead associated with the ith backbone network layer, while t_{b_i} represents the computation overhead attributable to the ith branch exit network.

Eq. (10) illustrates the optimization goal of the multi-exit DNN as the aggregate of the anticipated execution times, denoted by $target_{b_i}$.

$$\begin{aligned} Target_1 = &\min_{i\in[1,n]} \sum_{i=1}^{n} (\mu_i \times p_i \times target_{b_i}) \\ s.t. \quad & X_{l_i}[0] \oplus X_{l_i}[1] = 1 \\ & X_{b_i}[0] \oplus X_{b_i}[1] = 1 \end{aligned} \tag{10}$$

Confronted with a multi-exit DNN encompassing n branch exit networks, the solution space of Eq. (10) escalates exponentially to $O(2^n)$. Such complexity is unacceptable for tasks sensitive to latency.

4.3. Acceleration algorithm

Based on the preliminary experimental results presented in the previous section, this section proposes a heuristic pruning algorithm with the structure of multi-exit DNN and the prior exit probability P. This algorithm effectively discards a majority of potential strategy combinations and efficiently identifies the optimal task allocation strategy. The heuristic pruning algorithm is displayed in Algorithm 1, and its time complexity is $O(n)$, facilitating the determination of the optimal strategy for task allocation.

Algorithm 1 multi-exit DNN task allocation algorithm based on heterogeneous processor

Input: Information of backbone network layers and branch exit networks I_l, I_b; power consumption mode e; exit probability of branch exits p; transmission delay of intermediate data t_B^{trans}.
Output: Task allocation status of branch exit networks X_B.

```
for I_{l_i} ∈ I_L do
    Compute t_{l_i}^{cpu} and t_{l_i}^{gpu} with Eq. (4).
end for
for I_{b_i} ∈ I_B do
    Compute t_{b_i}^{cpu} and t_{b_i}^{gpu} with Eq. (4).
end for
for i = 1, 2, ..., n do
    if t_{b_i}^{cpu} + t_{b_i}^{trans} ≤ t_{b_i}^{gpu} / p_i || t_{b_i}^{cpu} + t_{b_i}^{trans} ≤ t_{b_i}^{gpu} + (1 − p_i) × t_{l_{i+1}}^{gpu} then
        X_{b_i} ← [0, 1]
    else
        X_{b_i} ← [1, 0]
    end if
end for
return X_B
```

By default, the inference tasks of the backbone network layer are consigned to the GPU. In Fig. 6, we define the start time as the moment when the ith exit point is reached, subsequent to the computation of the ith backbone network layer, and the end time as the instance when the $i+1$th backbone network layer has been computed. T_{cut} indicates the time interval between the end and start times.

① For $X_{b_i} = [0,1]$, with the CPU executing the inference tasks for the ith branch exit network, the GPU may persist in performing the inference tasks for the $i+1$th backbone network. As depicted in Fig. 7(a), $T_{cut}^{01} = p_i \times (t_{b_i}^{cpu} + t_{b_i}^{trans}) + (1-p_i) \times \max t_{b_i}^{cpu} + t_{b_i}^{trans}, t_{l_{i+1}}^{gpu}$.

② For $X_{b_i} = [1,0]$, the GPU is restricted to executing the inference tasks of the branch exit network and the backbone network in sequence. As illustrated in Fig. 7(b), $T_{cut}^{10} = p_i \times t_{b_i}^{gpu} + (1-p_i) \times (t_{b_i}^{gpu} + t_{l_{i+1}}^{gpu})$.

$$\begin{aligned} T_{cut}^{01} - T_{cut}^{10} =& p_i \times (t_{b_i}^{cpu} + t_{b_i}^{trans} - t_{b_i}^{gpu}) + \\ & (1-p_i) \times (\max\{t_{b_i}^{cpu} + t_{b_i}^{trans}, t_{l_{i+1}}^{gpu}\} - t_{b_i}^{gpu} - t_{l_{i+1}}^{gpu}) \end{aligned} \tag{11}$$

When $t_{b_i}^{cpu} + t_{b_i}^{trans} \le t_{b_i}^{gpu}$, $\max\{t_{b_i}^{cpu} + t_{b_i}^{trans}, t_{l_{i+1}}^{gpu}\} \le \max\{t_{b_i}^{gpu}, t_{l_{i+1}}^{gpu}\} < t_{b_i}^{gpu} + t_{l_{i+1}}^{gpu}$, and $T_{cut}^{01} < T_{cut}^{10}$. It is more appropriate to place the inference task of the ith branch exit network on the CPU.

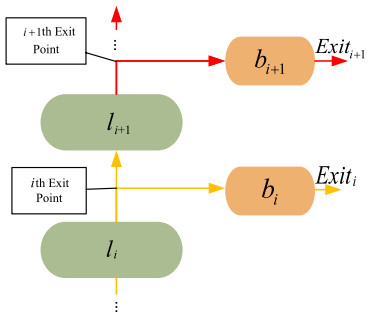
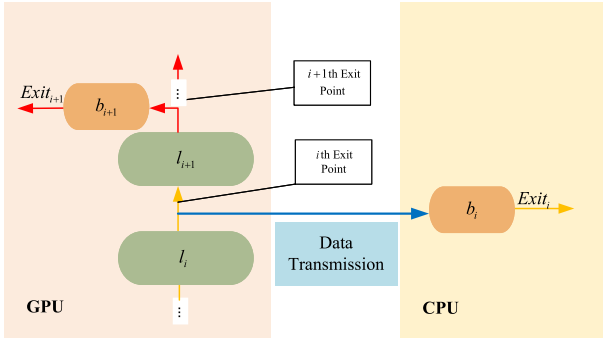
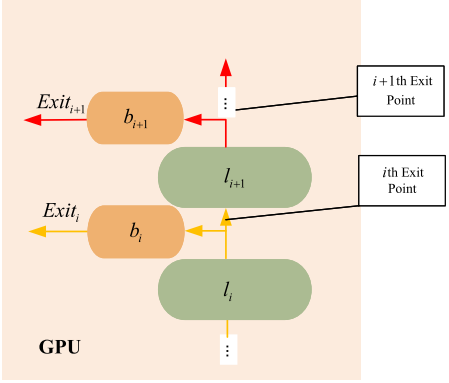


Fig. 6. The illustration of multi-exit DNNs task execution.



(a) b_i is performed on the CPU.



(b) b_i is performed on the GPU.

Fig. 7. The exit network execution on different processors.

When $t_{b_i}^{cpu} + t_{b_i}^{trans} > t_{b_i}^{gpu}$, we need to discuss the relationship between $t_{b_i}^{cpu} + t_{b_i}^{trans}$ and $t_{l_{i+1}}^{gpu}$.

(1) $t_{b_i}^{cpu} + t_{b_i}^{trans} \le t_{l_{i+1}}^{gpu}$:

$$\begin{aligned} T_{cut}^{01} - T_{cut}^{10} =& p_i \times (t_{b_i}^{cpu} + t_{b_i}^{trans} - t_{b_i}^{gpu}) - (1-p_i) \times t_{b_i}^{gpu} \\ =& p_i \times (t_{b_i}^{cpu} + t_{b_i}^{trans}) - t_{b_i}^{gpu} \end{aligned} \tag{12}$$

As shown in Eq. (12). When $t_{b_i}^{cpu} + t_{b_i}^{trans} \le \frac{t_{b_i}^{gpu}}{p_i}$, $T_{cut}^{01} - T_{cut}^{10} < 0$, and when $t_{b_i}^{cpu} + t_{b_i}^{trans} > \frac{t_{b_i}^{gpu}}{p_i}$, $T_{cut}^{01} - T_{cut}^{10} > 0$.

(2) $t_{b_i}^{cpu} + t_{b_i}^{trans} > t_{l_{i+1}}^{gpu}$:

$$\begin{aligned} T_{cut}^{01} - T_{cut}^{10} =& (p_i + 1 - p_i) \times (t_{b_i}^{cpu} + t_{b_i}^{trans} - t_{b_i}^{gpu}) - (1-p_i) \times t_{l_{i+1}}^{gpu} \\ =& t_{b_i}^{cpu} + t_{b_i}^{trans} - t_{b_i}^{gpu} - (1-p_i) \times t_{l_{i+1}}^{gpu} \end{aligned} \tag{13}$$

As shown in Eq. (13). When $t_{b_i}^{cpu} + t_{b_i}^{trans} \le t_{b_i}^{gpu} + (1-p_i) \times t_{l_{i+1}}^{gpu}$, $T_{cut}^{01} - T_{cut}^{10} \le 0$, and when $t_{b_i}^{cpu} + t_{b_i}^{trans} > t_{b_i}^{gpu} + (1-p_i) \times t_{l_{i+1}}^{gpu}$, $T_{cut}^{01} - T_{cut}^{10} > 0$.

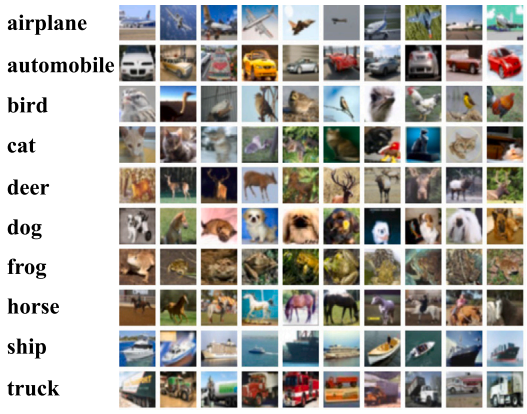


Fig. 8. CIFAR-10 dataset.

Table 4
The relevant parameters of Jetson Nano.

Property	High power	Low power
Power	10 W	5 W
Number of CPU cores	4	2
Maximum clock frequency of CPU (MHz)	1479	918
Maximum clock frequency of GPU (MHz)	921	640

As a conclusion, when $t_{b_i}^{cpu}+t_{b_i}^{trans} \le t_{b_i}^{gpu}+(1-p_i)\times t_{l_{i+1}}^{gpu}$ or $t_{b_i}^{cpu}+t_{b_i}^{trans} \le \frac{t_{b_i}^{gpu}}{p_i}$, it is more appropriate to place the inference task of the ith branch exit network on the CPU. Otherwise, it is more appropriate to place the inference task of the ith branch exit network on the GPU.

5. Experiment

To verify the effectiveness of the CIAHP, a series of comparisons were made with baselines. This section first presents the settings used for experiment. Then, we changed the traditional DNNs to multi-exit DNNs and used the test set to obtain the estimated prior exit probability. We perform a series of experiments on model inference tasks using a terminal device, considering various power consumption modes. The average value derived from these experiments is considered as the obtained experimental outcomes. Finally, we analyzed the experimental results and gave the corresponding conclusions.

5.1. Setup

5.1.1. Datasets and training model

For this investigation, we selected the CIFAR-10 dataset [37] as illustrated in Fig. 8. This dataset comprises 60,000 RGB images in total, with 50,000 serving as training images, and the remaining 10,000 as testing images. These test images serve as input samples to substantiate the effectiveness of CIAHP.

To verify the efficacy of CIAHP, we employed the NVIDIA Jetson Nano equipped with heterogeneous processor. As depicted in Fig. 9, the NVIDIA Jetson Nano is fitted with a Maxwell GPU, a quad-core ARM Cortex-A57 64-bit CPU, 4 GB of LPDDR4 memory, and a 16 GB external SD card. This intelligent terminal device operates in two power consumption modes, specifically 10 W and 5 W, which offer diverse computational capabilities for CPUs and GPUs. Table 4 exhibits the pertinent parameters of Jetson Nano.

The PyTorch deep learning framework was employed to fabricate the experimental system, as portrayed in Fig. 10. This system encapsulates three modules: task scheduler, task workers, and result collector. The task scheduler oversees task distribution amongst CPUs and GPUs. The task workers, both CPU and GPU, execute inference tasks, and potential intermediate data transfer might occur between these workers.

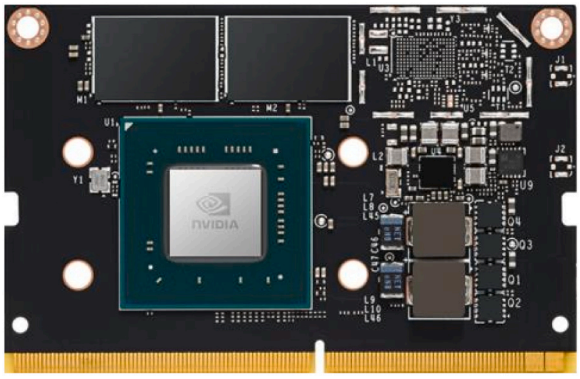


Fig. 9. NVIDIA Jetson Nano.

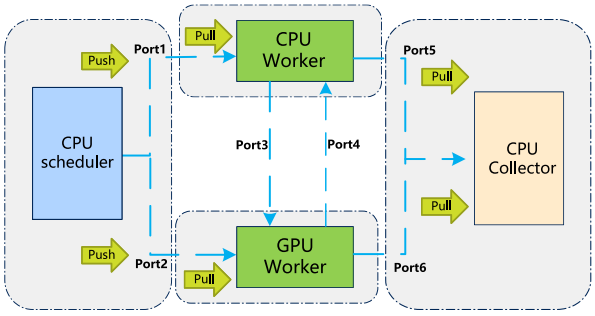


Fig. 10. The system architecture.

Table 5
The exit probability of branch exits.

Model	$P = p_0, p_1, p_2, \ldots, p_{n-1}, p_n$
ME-AlexNet	$P = 0, 0.7661, 0.7449, 0.6156, 0.6448, 0.8791$
ME-VGG	$P = 0, 0.8223, 0.8295, 0.6810, 0.7104, 0.9136$

Upon completion of tasks, the worker transmits the results to the result collector. The latter module is responsible for processing statistical tasks, collecting inference latency, and recording recognition results.

We used AlexNet [1] and VGG16 [38] models and designed simplified Multi-Exit variants of AlexNet (ME-AlexNet) and VGG (ME-VGG) based on the CIFAR-10 dataset. Both ME-AlexNet and ME-VGG are furnished with five branch exits each. Table 5 illustrates the estimated prior exit probability P of each branch exit, ascertained using the test set.

5.1.2. Compared methods

Firstly, we compared and analyzed the inference latency and accuracy of traditional DNNs and multi-exit DNNs to demonstrate the effectiveness of the multi-exit architecture. Secondly, we evaluated the acceleration performance of disparate inference algorithms predicated on multi-exit DNNs. Lastly, we assessed the efficiency of various inference algorithms utilizing complex image data as input, thereby affirming the acceleration performance of the CIAHP algorithm predicated on complex input.

Specifically, we compared and analyzed the performance of four algorithms that disregard heterogeneous processor cooperation. These include: (1) **CPU-Only**. Place all tasks on the CPU. (2) **GPU-Only**. Place all tasks on the GPU. (3) **0-1 pattern**. A part of the image recognition task is executed on the CPU and the other part is executed on the GPU. (4) **DeepX** [28]. DeepX divides the deep learning model into multiple modules that are executed on the GPU and CPU in synchronous manner according to the characteristics of each module, which only utilizes one processor for a task at a time. Our experiments are conducted in both

Table 6
The complexity distribution of test data set.

Model	b1	b2	b3	b4	b5
ME-AlexNet	7661	1379	266	163	375
ME-VGG	8223	1297	198	110	141

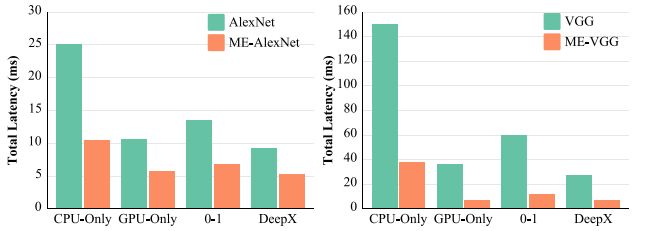


Fig. 11. Comparison of inference performance between multi-exit DNN models and traditional models (low-power).

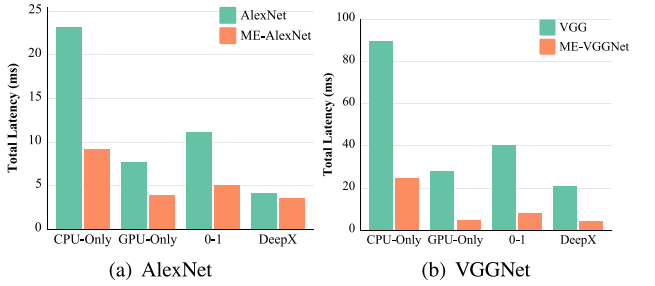


Fig. 12. Comparison of inference performance between multi-exit DNN models and traditional models (high-power).

high and low power consumption modes, and the final results are the average inference latency across multiple tasks.

5.2. Experimental results and analysis

5.2.1. The exit distribution of data

Initially, we undertook an analysis of the complexity distribution of the test set. Table 6 enumerates $b_1, b_2, \ldots, b_5$, denoting the five exits of the model, arranged in ascending order from the shallowest to the deepest layers. A majority of the samples were found to exit at the shallowest exits, whereas only a handful were found to exit at the final layer, which the model identifies as complex samples. As demonstrated in the ensuing experimental results, these complex samples encompass the entirety of the backbone network as well as all branch exit networks, hence yielding the relatively higher inference latency compared to other samples. Contrary to Table 5, which represents the probability of samples exiting at the corresponding exit, Table 6 showcases the complexity distribution of the samples in the test set.

5.2.2. Performance comparison

Initially, we performed a comparison between multi-exit neural networks and conventional neural networks, focusing on their respective inference latency under various power consumption modes. Subsequently, we evaluated the effectiveness of the CIAHP algorithm. Further, we conducted a separate analysis for instances where complex samples were input, and we validated the acceleration impact conferred by the application of the CIAHP algorithm.

The impact of multi-exit architecture: Under the low-power mode, ME-AlexNet and ME-VGG are capable of extracting sufficient features for most samples at the shallow layers of the network, thereby enabling early termination of the inference process. This results in a speedup of 3.0× and 5.0× for AlexNet and VGGNet, respectively. Similar results are observed under the high-power mode. The results are displayed in Fig. 11 and Fig. 12. The multi-exit architecture also marginally enhances the accuracy of the traditional models, as depicted in Table 7.

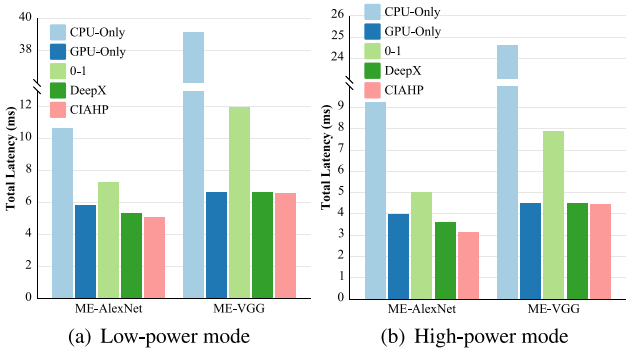


Fig. 13. Acceleration performance comparison of each algorithm using multi-exit DNN.

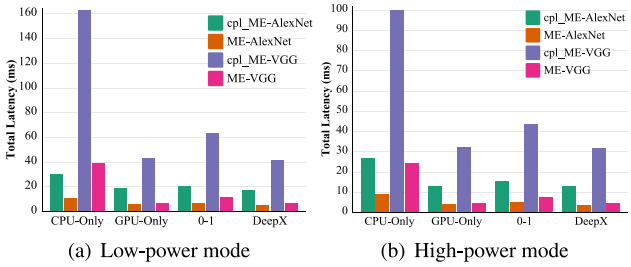


Fig. 14. Comparison of average inference latency and inference latency of complex samples.

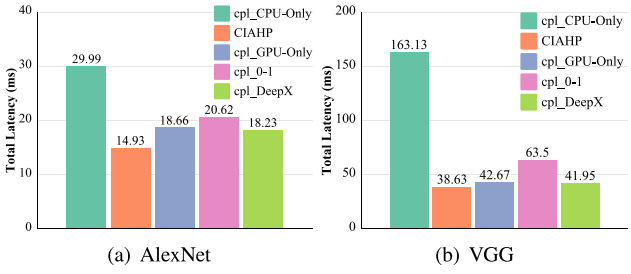


Fig. 15. Acceleration performance comparison of acceleration algorithms with complex samples (low-power).

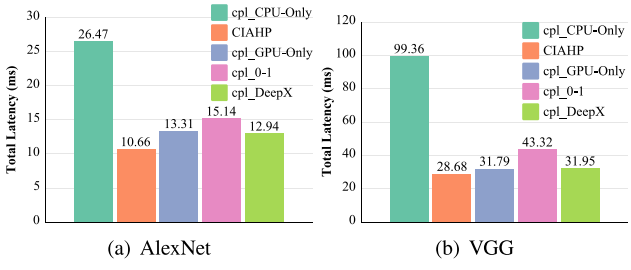


Fig. 16. Acceleration performance comparison of acceleration algorithms with complex samples (high-power).

The effectiveness of the CIAHP: We compared the acceleration performance of CIAHP and traditional acceleration algorithms using multi-exit DNN. According to Fig. 13, CIAHP exhibits the highest performance. Inference latency is lower in CIAHP due to the fact that

Table 7
Comparison of the accuracy of different models.

Model	Accuracy
AlexNet	75.68%
ME-AlexNet	76.18%
VGG	82.86%
ME-VGG	83.22%

GPUs and CPUs can be used in parallel to perform inference tasks. In comparison to the Gpu-Only and DeepX approaches, the CIAHP framework demonstrates a speedup of 1.2× and 1.1× on the ME-AlexNet model, respectively. However, because the multi-exit DNN allows most samples to exit early, there is no obvious difference between CIAHP and GPU-Only when comparing the acceleration effect for ME-VGG.

The impact of complex samples: We examined the time overhead of different acceleration algorithms when processing complex samples. As shown in Fig. 14(a), the inference latency of the models (cpl_ME-AlexNet and cpl_ME-VGG) whose input is complex samples increases sharply compared with the average inference latency in low-power mode. Because all backbone networks and branch networks are covered by the inference path for complex samples. As shown in Fig. 14(b), there are similar results in high power mode.

Based on the complex samples mentioned in Table 6, this paper evaluates the inference performance of the CIAHP algorithm and other algorithms. The results are shown in Fig. 15 and Fig. 16. Compared to GPU-Only, CIAHP improves performance by approximately 12.4% and 11.1% on ME-AlexNet and ME-VGG respectively, similar results are observed in the high-power mode. In the same way, compared to DeepX, CIAHP improves performance by approximately 12.2% and 11.0% on ME-AlexNet and ME-VGG respectively. CIAHP accounts for the collaborative nature of heterogeneous processors, thereby allowing the inference tasks of the backbone and branches of a multi-exit DNN to be executed simultaneously by the GPU and CPU. The latency caused by complex samples can be offset due to the ability of heterogeneous processors to perform tasks in parallel.

DeepX demonstrates the capability to allocate processors based on the distinctive attributes of various network layers. Consequently, this approach yields superior acceleration compared to GPU-Only. Nevertheless, DeepX facilitates the acceleration of inference tasks on heterogeneous processors through a synchronous procedure, wherein only a single processor carries out an inference task at any given moment, rendering the remaining processors inactive. Therefore, CIAHP exhibits a distinct advantage over DeepX in terms of the attained speedup results for inference tasks. However, this advantageous trend exhibits a diminishing pattern when applied to AlexNet, and VGGNet, respectively. To validate this trend, we conducted a supplementary experiment utilizing the LeNet model [39], which exhibits a notably simpler architecture when compared to AlexNet and VGG. The experimental findings indicate a significant superiority of CIAHP over DeepX in terms of LeNet acceleration. This phenomenon can be attributed to inter-processor communication, as the complexity of the model increases, resulting in elevated costs and frequency of communication among processors in CIAHP.

6. Conclusions

In this paper, the problem of inference acceleration based on heterogeneous processors is investigated. We propose CIAHP, a heuristic-guided inference acceleration mechanism for intelligent terminal devices with heterogeneous processors. The CIAHP consists of two stages, including the deep neural network computation time prediction model and the multi-exit DNN task allocation algorithm based on heterogeneous processors. An optimal task allocation strategy can be promptly identified using the CIAHP. Experiments have shown that the multi-exit DNN inference acceleration mechanism based on heterogeneous processors proposed in this paper can ensure the stable operation of devices under a variety of power consumption modes. Compared to traditional inference acceleration mechanisms, this mechanism reduces the average inference latency and effectively alleviates the performance degradation of multi-exit DNNs with complex inputs. However, our methodology fails to consider the memory resource competition between the GPU and CPU, thereby resulting in a decline in the efficiency of inference. Considering the utilization of memory and the competition for memory between GPUs and CPUs in inference tasks, would be promising research directions.

CRediT authorship contribution statement

Jinghui Zhang: Conceptualization, Methodology. **Weilong Xin:** Visualization, Software, Writing – review & editing. **Dingyang Lv:** Visualization, Software. **Jiawei Wang:** Writing – review & editing. **Guangxing Cai:** Methodology, Software, Writing – original draft. **Fang Dong:** Supervision, Resources, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The authors do not have permission to share data.

Acknowledgments

This work is supported by National Natural Science Foundation of China under Grants No. 61972085, 62232004, Jiangsu Provincial Key Laboratory of Network and Information Security, China under Grants No. BM2003201, Key Laboratory of Computer Network and Information Integration of Ministry of Education of China under Grants No. 93K-9, the University Synergy Innovation Program of Anhui Province, China No. GXXT-2020-012, and partially supported by Collaborative Innovation Center of Novel Software Technology and Industrialization, the Fundamental Research Funds for the Central Universities, CCF-Baidu Open Fund, China (No. 2021PP15002000) and Future Network Scientific Research Fund Project, China (FNSRFP-2021-YB-02). We also thank the Big Data Computing Center of Southeast University for providing the experiment environment and computing facility.

References

[1] A. Krizhevsky, I. Sutskever, G.E. Hinton, Imagenet classification with deep convolutional neural networks, Commun. ACM 60 (6) (2017) 84–90.
[2] Q. Zhang, H. Sun, X. Wu, H. Zhong, Edge video analytics for public safety: A review, Proc. IEEE 107 (8) (2019) 1675–1696.
[3] G.V. Reddy, K. Deepika, L. Malliga, D. Hemanand, C. Senthilkumar, S. Gopalakrishnan, Y. Farhaoui, Human action recognition using difference of Gaussian and difference of wavelet, Big Data Min. Anal. 6 (3) (2023) 336–346.
[4] S. Khetavath, N.C. Sendhilkumar, P. Mukunthan, S. Jana, S. Gopalakrishnan, L. Malliga, S.R. Chand, Y. Farhaoui, An intelligent heuristic manta-ray foraging optimization and adaptive extreme learning machine for hand gesture image recognition, Big Data Min. Anal. 6 (3) (2023) 321–335.
[5] D. Chatzopoulos, C. Bermejo, Z. Huang, P. Hui, Mobile augmented reality survey: From where we are to where we go, IEEE Access 5 (2017) 6917–6950.
[6] A.G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, H. Adam, Mobilenets: Efficient convolutional neural networks for mobile vision applications, 2017, arXiv preprint arXiv:1704.04861.
[7] X. Xu, H. Li, W. Xu, Z. Liu, L. Yao, F. Dai, Artificial intelligence for edge service optimization in internet of vehicles: A survey, Tsinghua Sci. Technol. 27 (2) (2022) 270–287, http://dx.doi.org/10.26599/TST.2020.9010025.
[8] Z. Huang, F. Dong, D. Shen, J. Zhang, H. Wang, G. Cai, Q. He, Enabling low latency edge intelligence based on multi-exit DNNs in the wild, in: 2021 IEEE 41st International Conference on Distributed Computing Systems, ICDCS, 2021, pp. 729–739.

[9] S. Teerapittayanon, B. McDanel, H.-T. Kung, Distributed deep neural networks over the cloud, the edge and end devices, in: 2017 IEEE 37th International Conference on Distributed Computing Systems, ICDCS, IEEE, 2017, pp. 328–339.
[10] A. Kalay, A. Fisne, Poster: Edge computing for deep learning-based sensor multi-target detection, in: 2022 IEEE/ACM 7th Symposium on Edge Computing, SEC, IEEE, 2022, pp. 286–288.
[11] J. Jeon, J.-K. Kang, Y. Kim, Filter pruning method for inference time acceleration based on YOLOX in edge device, in: 2022 19th International SoC Design Conference, ISOCC, IEEE, 2022, pp. 354–355.
[12] P. Regulation, General data protection regulation, Intouch 25 (2018) 1–5.
[13] A. Heidari, N.J. Navimipour, M.A.J. Jamali, S. Akbarpour, A green, secure, and deep intelligent method for dynamic IoT-edge-cloud offloading scenarios, Sustain. Comput. Inf. Syst. (2023) 100859.
[14] N. Sanil, V. Rakesh, R. Mallapur, M.R. Ahmed, et al., Deep learning techniques for obstacle detection and avoidance in driverless cars, in: 2020 International Conference on Artificial Intelligence and Signal Processing, AISP, IEEE, 2020, pp. 1–4.
[15] C. Hu, W. Bao, D. Wang, F. Liu, Dynamic adaptive DNN surgery for inference acceleration on the edge, in: IEEE INFOCOM 2019-IEEE Conference on Computer Communications, IEEE, 2019, pp. 1423–1431.
[16] S. Laskaridis, S.I. Venieris, M. Almeida, I. Leontiadis, N.D. Lane, SPINN: synergistic progressive inference of neural networks over device and cloud, in: Proceedings of the 26th Annual International Conference on Mobile Computing and Networking, 2020, pp. 1–15.
[17] F. Dong, H. Wang, D. Shen, Z. Huang, Q. He, J. Zhang, L. Wen, T. Zhang, Multi-exit DNN inference acceleration based on multi-dimensional optimization for edge intelligence, IEEE Trans. Mob. Comput. (2022).
[18] D. Kang, J. Oh, J. Choi, Y. Yi, S. Ha, Scheduling of deep learning applications onto heterogeneous processors in an embedded device, IEEE Access 8 (2020) 43980–43991.
[19] F. Zhang, J. Zhai, B. Wu, B. He, W. Chen, X. Du, Automatic irregularity-aware fine-grained workload partitioning on integrated architectures, IEEE Trans. Knowl. Data Eng. 33 (3) (2019) 867–881.
[20] Y. Kim, J. Kim, D. Chae, D. Kim, J. Kim, μLayer: Low latency on-device inference using cooperative single-layer acceleration and processor-friendly quantization, in: Proceedings of the Fourteenth EuroSys Conference 2019, 2019, pp. 1–15.
[21] S. Zhang, W. Cui, Q. Chen, Z. Zhang, Y. Guan, J. Leng, C. Li, M. Guo, PAME: precision-aware multi-exit DNN serving for reducing latencies of batched inferences, in: Proceedings of the 36th ACM International Conference on Supercomputing, 2022.
[22] W. Ju, W. Bao, D. Yuan, L. Ge, B.B. Zhou, Learning early exit for deep neural network inference on mobile devices through multi-armed bandits, in: 2021 IEEE/ACM 21st International Symposium on Cluster, Cloud and Internet Computing (CCGrid), 2021, pp. 11–20.
[23] S. Chetlur, C. Woolley, P. Vandermersch, J. Cohen, J. Tran, B. Catanzaro, E. Shelhamer, cudnn: Efficient primitives for deep learning, 2014, arXiv preprint arXiv:1410.0759.
[24] N.P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers, et al., In-datacenter performance analysis of a tensor processing unit, in: Proceedings of the 44th Annual International Symposium on Computer Architecture, 2017, pp. 1–12.
[25] E. Jeong, J. Kim, S. Ha, TensorRT-based framework and optimization methodology for deep learning inference on jetson boards, ACM Trans. Embedded Comput. Syst. (TECS) 21 (5) (2022) 1–26.
[26] Y. Xiang, H. Kim, Pipelined data-parallel CPU/GPU scheduling for multi-DNN real-time inference, in: 2019 IEEE Real-Time Systems Symposium, RTSS, IEEE, 2019, pp. 392–405.
[27] J.S. Jeong, J. Lee, D. Kim, C. Jeon, C. Jeong, Y. Lee, B.-G. Chun, Band: coordinated multi-DNN inference on heterogeneous mobile processors, in: Proceedings of the 20th Annual International Conference on Mobile Systems, Applications and Services, 2022, pp. 235–247.
[28] N.D. Lane, S. Bhattacharya, P. Georgiev, C. Forlivesi, L. Jiao, L. Qendro, F. Kawsar, Deepx: A software accelerator for low-power deep learning inference on mobile devices, in: 2016 15th ACM/IEEE International Conference on Information Processing in Sensor Networks (IPSN), IEEE, 2016, pp. 1–12.
[29] S. Teerapittayanon, B. McDanel, H.-T. Kung, Branchynet: Fast inference via early exiting from deep neural networks, in: 2016 23rd International Conference on Pattern Recognition, ICPR, IEEE, 2016, pp. 2464–2469.
[30] H. Wang, G. Cai, Z. Huang, F. Dong, ADDA: Adaptive distributed DNN inference acceleration in edge computing environment, in: 2019 IEEE 25th International Conference on Parallel and Distributed Systems, ICPADS, IEEE, 2019, pp. 438–445.
[31] E. Li, L. Zeng, Z. Zhou, X. Chen, Edge AI: On-demand accelerating deep neural network inference via edge computing, IEEE Trans. Wireless Commun. 19 (1) (2019) 447–457.
[32] S. Nath, J. Wu, Deep reinforcement learning for dynamic computation offloading and resource allocation in cache-assisted mobile edge computing systems, Intell. Converg. Netw. 1 (2) (2020) 181–198.
[33] G.S. Rahman, T. Dang, M. Ahmed, Deep reinforcement learning based computation offloading and resource allocation for low-latency fog radio access networks, Intell. Converg. Netw. 1 (3) (2020) 243–257.
[34] R. Bi, Q. Liu, J. Ren, G. Tan, Utility aware offloading for mobile-edge computing, Tsinghua Sci. Technol. 26 (2) (2020) 239–250.
[35] M. Zhou, B. Zhou, H. Wang, F. Dong, W. Zhao, Dynamic path based DNN synergistic inference acceleration in edge computing environment, in: 2021 IEEE 27th International Conference on Parallel and Distributed Systems, ICPADS, 2021, pp. 567–574.
[36] Y. Jia, Learning Semantic Image Representations at a Large Scale, University of California, Berkeley, 2014.
[37] A. Krizhevsky, G. Hinton, et al., Learning Multiple Layers of Features from Tiny Images, Toronto, ON, Canada, 2009.
[38] K. Simonyan, A. Zisserman, Very deep convolutional networks for large-scale image recognition, 2014, arXiv preprint arXiv:1409.1556.
[39] Y. LeCun, L. Bottou, Y. Bengio, P. Haffner, Gradient-based learning applied to document recognition, Proc. IEEE 86 (1998) 2278–2324.